

ABSTRACT

Road Safety is an integral part of modern roads. Therefore, speed limits are given for a particular road depending on the quality of the road and the how prone the road is to accidents. Speed Cameras are set up at intervals of the road to catch speed-limit violators. Just like any other technology, speed cameras have progressed over the years.

This project presents a comprehensive solution for real-time traffic monitoring and vehicle speed tracking using computer vision techniques with the OpenCV library. The system processes video streams, applies background subtraction and morphological operations to detect and track vehicles within a defined region of interest (ROI). It calculates and annotates vehicle speeds and visually identifies vehicles exceeding a predetermined speed limit. In cases of speed limit violations, the system captures images of the offending vehicles for further analysis. To conclude, the project provides a detailed summary of the collected data, including the total count of tracked vehicles and those that have exceeded the speed limit. This versatile solution can be adapted and configured to suit a wide range of video sources and data recording requirements.

1.INTRODUCTION

1.1 Motivation

The motivation behind this project lies in addressing critical issues plaguing transportation systems worldwide, including air pollution, accidents, and traffic congestion. By implementing Intelligent Transportation Systems (ITS), we aim to modernize and enhance road safety through real-time vehicle speed estimation using non-intrusive methods such as video analysis. With the alarming rise in road accidents, especially in countries like India with burgeoning vehicle populations, there's an urgent need for effective speed detection systems. This project seeks to fill this gap by developing a cost-effective solution capable of accurately monitoring vehicle speeds, enforcing speed limits, and generating e-challans for offenders. By collaborating with public and private road safety agencies, we aim to contribute to the overarching goal of reducing fatalities caused by speeding vehicles. Ultimately, this project aligns with global efforts to prioritize road safety and underscores the necessity for innovative traffic enforcement technologies to safeguard lives and improve overall transportation efficiency.

1.2 Problem Definition:

Vehicle tracking is the process of locating a moving vehicle using a camera. Capture vehicle in video sequence from surveillance camera is demanding application to improve tracking performance. This technology is increasing the number of applications such as traffic control, traffic monitoring, traffic flow, security etc. The estimated cost using this technology will be very less. Video and image processing has been used for traffic surveillance, analysis and monitoring of traffic conditions in many cities and urban areas. Various methods for speed estimation are proposed in recent years. All approaches attempt to increase accuracy and decrease cost of hardware implementation. The aim is to build an automatic system that can accurately localise and track the speed of any vehicles that appear in aerial video frames.

1.3 Objectives:

1. Automated Speed Detection:

Develop an automated system capable of accurately detecting and monitoring the speed of vehicles in real-time.

2. Computer Vision Techniques:

Utilize computer vision techniques, including background subtraction, contour analysis, and object tracking, to process video data and extract relevant information about vehicle speed.

3. Real-Time Processing:

Implement algorithms that can operate in real-time to ensure timely and efficient speed detection without causing delays in processing.

4. Precision and Reliability:

Enhance the precision and reliability of speed detection by addressing challenges such as varying weather conditions, different vehicle sizes, and other environmental factors.

5. Object Tracking:

Implement an object tracking mechanism to monitor the movement of vehicles across consecutive frames, allowing for the calculation of accurate speed metrics.

6. Integration with Traffic Infrastructure:

Design the system with the flexibility to integrate seamlessly with existing traffic infrastructure, making it adaptable for deployment in urban environments.

7. Data Logging and Reporting:

Develop a system to log speed data for each vehicle, enabling the generation of comprehensive reports. This data can be used for traffic management, law enforcement, and urban planning.

8. Adaptability to Various Environments:

Ensure that the system is adaptable to diverse environments, considering factors such as different road types, lighting conditions, and vehicle densities.

9. Validation and Testing:

Conduct thorough validation and testing of the system in real-world scenarios to verify its accuracy, reliability, and robustness under different conditions.

By addressing these objectives, the project aims to contribute to the development of a state-of-the-art automated vehicle speed detection system that can enhance road safety, optimize traffic flow, and provide valuable data for urban planning and transportation management.

2. LITERATURE REVIEW

According to our knowledge there exist only limited surveys and research papers related to detecting the speed of the vehicle. They use different techniques to detect the speed of the vehicle. Some researchers use ramp metering technique and harmonization algorithm to detect speed. Vehicle speed detection is a major cause to research in the field of Intelligent Transportation System (ITS). Many researchers applied various techniques to detect the vehicle and to measure the speed of the vehicle.

Mostly all the techniques are based on computer software and the hardware equipment. As the number of vehicles are increasing day by day it is becoming difficult to manually control and monitor traffic. Moreover, it is costly, time consuming and sometimes it is impossible to control the traffic. To overcome all this problem cameras are installed on highway, road, for monitoring the vehicle passing on the roads. Several parameters like fog, dust, rainfall, snowfall, light density or blur scenario can influence the quality of captured images. The speed of the vehicle is measured using the frames of an input video. This method proposes a real time application based on real time response. The thought of using video camera technique to find the speed of the vehicle is suggested to enhance the current method of speed detection which is mostly based on radar hardware. The radar technique is having various disadvantages like the cost of equipment is very high. So, there is a need to shift from this method to reduce the cost of investment. Video camera technique can be used as an alternative of radar system. This video camera technique does not require specific hardware. It is mostly based on software components. We require a camera to capture a video and a computer to detect a speed.

As the population is increasing continuously people encounter many problems. One of the main problems is traffic accident. Since this problem is affecting adversely in daily life, so there is a need to find the solution of this problem or to reduce the effects of this problem. For traffic enforcement observation of vehicles speed is important. Traditionally, speed of the vehicles is calculated through radar gun or radar detector. In intelligent transportation system, vehicle detection and speed recognition play an important role. There are various approaches to detect the speed of the vehicle. Technologies like helicopter mobile radar, artificial vision or license plate recognition technology. It helps to increase the road safety and to reduce the number of over-speeding vehicles. There is no regulatory system to find the unpredictability or exact speed estimation. Sensors are less accurate so that their use is reduced to detect the speed. We can also use Internet-of-Things (IoT) devices to detect the speed.

To overcome the problem of road accidents various automobile companies, try to give a speed control technique to keep vehicle safe. IoT technology can send an alert about over-speeding of the vehicle and also helps in drunk and drive case too. It also provides solution for rash driving. IoT devices uses sensor to detect the speed and have a great accuracy. Some researchers also use doppler effect to detect the vehicle speed if over-speeding found then uses digital image processing to mark the license number. This system is having a great accuracy and better performance to find the speed of the vehicle. Some researchers use Arduino, Infrared (IR) sensors and jumper wires to find the vehicle speed. We have heard slogans like drive-slowly, over speeding kills etc. on road. It is a threat to the life of people and can put various lives in danger.

2.2. TECHNOLOGIES USING FOR VEHICLE SPEED TRACKING

Introduction to Vehicle Speed Tracking:

- Definition and significance of vehicle speed tracking.
- Importance in road safety, traffic management, and fleet monitoring.

Existing Speed Tracking Systems:

- Overview of current methods and technologies used for tracking vehicle speed.
- Comparison of different systems, such as radar-based, GPS-based, and camera-based speed tracking.
- Evaluation of their accuracy, reliability, and limitations.

GPS-Based Speed Tracking Systems:

- Detailed examination of GPS-based speed tracking systems.
- Discussion on how GPS technology works for speed measurement.
- Review of existing GPS-based speed tracking devices and their performance.

Sensor Technologies for Speed Measurement:

- Exploration of various sensor technologies used for speed measurement, such as radar, lidar, and ultrasonic sensors.
- Comparison of sensor accuracy, cost, and applicability in different environments.

Data Analysis and Processing:

- Discussion on data processing techniques used in speed tracking systems.
- Analysis of algorithms for filtering noise, smoothing data, and calculating accurate speed readings.

Integration with Vehicle Telematics Systems:

- Review of how speed tracking systems are integrated with vehicle telematics platforms.
- Examination of features like real-time monitoring, historical data analysis, and alerts for speeding violations.

Challenges and Future Directions:

- Identification of challenges in speed tracking, such as signal interference, accuracy issues, and legal constraints.
- Discussion on potential future advancements in speed tracking technology, including AI-based systems and predictive analytics.

Case Studies and Applications:

- Case studies highlighting successful implementations of vehicle speed tracking systems in real-world scenarios.
- Applications in law enforcement, transportation management, and commercial fleet operations.

3. ANALYSIS

3.1 Existing System:

The current method of vehicle speed measurement primarily relies on radar-based techniques, which can be costly to implement and maintain. Additionally, intrusive methods like inductive loop detectors are used on highways, but they are challenging to install and upkeep. These systems may not provide real-time speed estimation and lack the ability to capture visual data for further analysis or evidence.

3.2 Proposed System:

The proposed system aims to overcome the limitations of the existing methods by introducing a non-intrusive, video-based approach for real-time vehicle speed estimation. By leveraging video cameras installed along highways, the system will analyse the footage to accurately track and measure the speed of passing vehicles. This approach offers advantages such as cost-effectiveness, ease of implementation, and continuous monitoring capabilities. Furthermore, the system will include a web application to stream live video and maintain a record of vehicle speeds, facilitating efficient traffic enforcement and promoting road safety.

3.3 Software requirement specification

3.3.1 Purpose:

The purpose of this project is to develop an advanced Intelligent Transportation System (ITS) aimed at enhancing road safety through real-time vehicle speed estimation. By utilizing non-intrusive video-based methods, the system aims to accurately monitor vehicle speeds on highways, thereby reducing the incidence of accidents caused by speeding vehicles. The primary objective is to create a cost-effective and efficient solution that can effectively enforce speed limits, generate e-challans for offenders, and contribute to the overarching goal of improving road safety.

3.3.2 Scope:

The scope of this project encompasses the design and implementation of a real-time vehicle speed detection system using video analysis techniques. The system will involve the installation of video cameras along highways to capture footage of passing vehicles. Through image processing algorithms, the system will track and measure the speed of each vehicle accurately. Additionally, a web application will be developed to stream live video and maintain a record of vehicle speeds for further analysis and enforcement purposes. The project will focus on ensuring scalability, reliability, and cost-effectiveness to cater to the needs of various stakeholders, including public road safety agencies, private organizations, and government authorities.

3.3.3 Overall Description:

The proposed system will leverage advanced video analysis technology to estimate the speed of vehicles in real-time. Video cameras installed along highways will capture footage of passing vehicles, which will be processed using image processing algorithms to track and measure vehicle speeds accurately. The system will provide a user-friendly web application interface for streaming live video and maintaining a database of vehicle speed records. Additionally, the system will be equipped with features to enforce speed limits, generate e-challans for speeding offenders, and facilitate efficient traffic management. Overall, the project aims to develop a robust and cost-effective solution to improve road safety and enhance transportation efficiency.

4. DESIGN

4.1 UML DIAGRAMS

A UML Diagram is based on UML (Unified Modelling Language) with the purpose of visually representing a system along with its main actors, roles, actions, artifacts or classes, in order to better understand, alter, maintain, or document information about the system.

The UML diagrams are categorized into **structural diagrams**, **behavioural diagrams**, and also interaction **overview diagrams**.

Structural diagrams, behavioural diagrams, and interaction overview diagrams are types of diagrams used in software engineering and systems modelling to visually represent different aspects of a system or process. Here's a brief overview of each:

Structural Diagrams:

Structural diagrams focus on depicting the static structure of a system, showing its components and their relationships. They provide a blueprint of the system's architecture and are typically used in the early stages of system design. Some common types of structural diagrams include:

- 1. Class Diagrams:** Show the classes or objects in a system and their attributes and relationships.
- 2. Object Diagrams:** Provide a snapshot of instances of classes and their relationships at a specific point in time.
- 3. Component Diagrams:** Illustrate the physical components and their connections in a system.
- 4. Package Diagrams:** Display how classes or components are organized into packages or namespaces.
- 5. Composite Structure Diagrams:** Focus on the internal structure of a class or component, showing how it is composed of other elements.

Behavioural Diagrams:

Behavioural diagrams, on the other hand, focus on representing the dynamic behavior and interactions between the components of a system. They are used to describe the system's functionality and how it responds to various stimuli. Some common types of behavioral diagrams include:

- 1. Use Case Diagrams:** Describe the interactions between system actors (users) and the system to capture its functionality from a user's perspective.
- 2. Activity Diagrams:** Model the workflow or business processes within the system, showing the sequence of activities and decision points.
- 3. Sequence Diagrams:** Depict the interactions between objects or components over time, emphasizing the order in which messages are exchanged.
- 4. State Machine Diagrams:** Show the different states an object or component can be in and how it transitions between these states based on events.
- 5. Collaboration Diagrams (Communication Diagrams):** Illustrate the interactions between objects and the messages exchanged between them.

Interaction Diagrams:

Interaction Diagrams are a specialized type of UML (Unified Modelling Language) diagram that combines elements of both structural and behavioral modelling. These diagrams provide an overview of how different components or objects interact to achieve a specific use case or scenario. They can include elements like decision nodes, merge nodes, and activity diagrams to represent various interactions within the system.

USECASE DIAGRAM

A use case diagram provides a visual overview of how users interact with a system. It shows actors (users or external systems) and the tasks they can perform (use cases). It's a helpful tool for understanding system functionality and requirements, aiding communication with stakeholders.

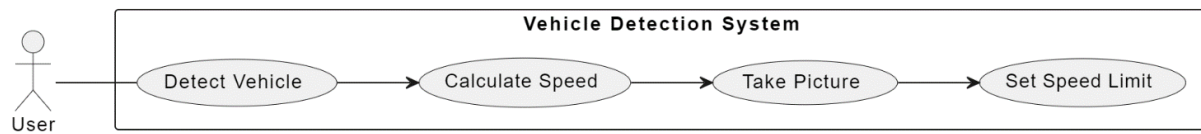


Figure 1 Use Case Diagram

ACTIVITY DIAGRAM

An activity diagram visually represents the flow of actions within a system or process. It shows the sequence of activities, decisions, and transitions involved in completing tasks or achieve a goal.

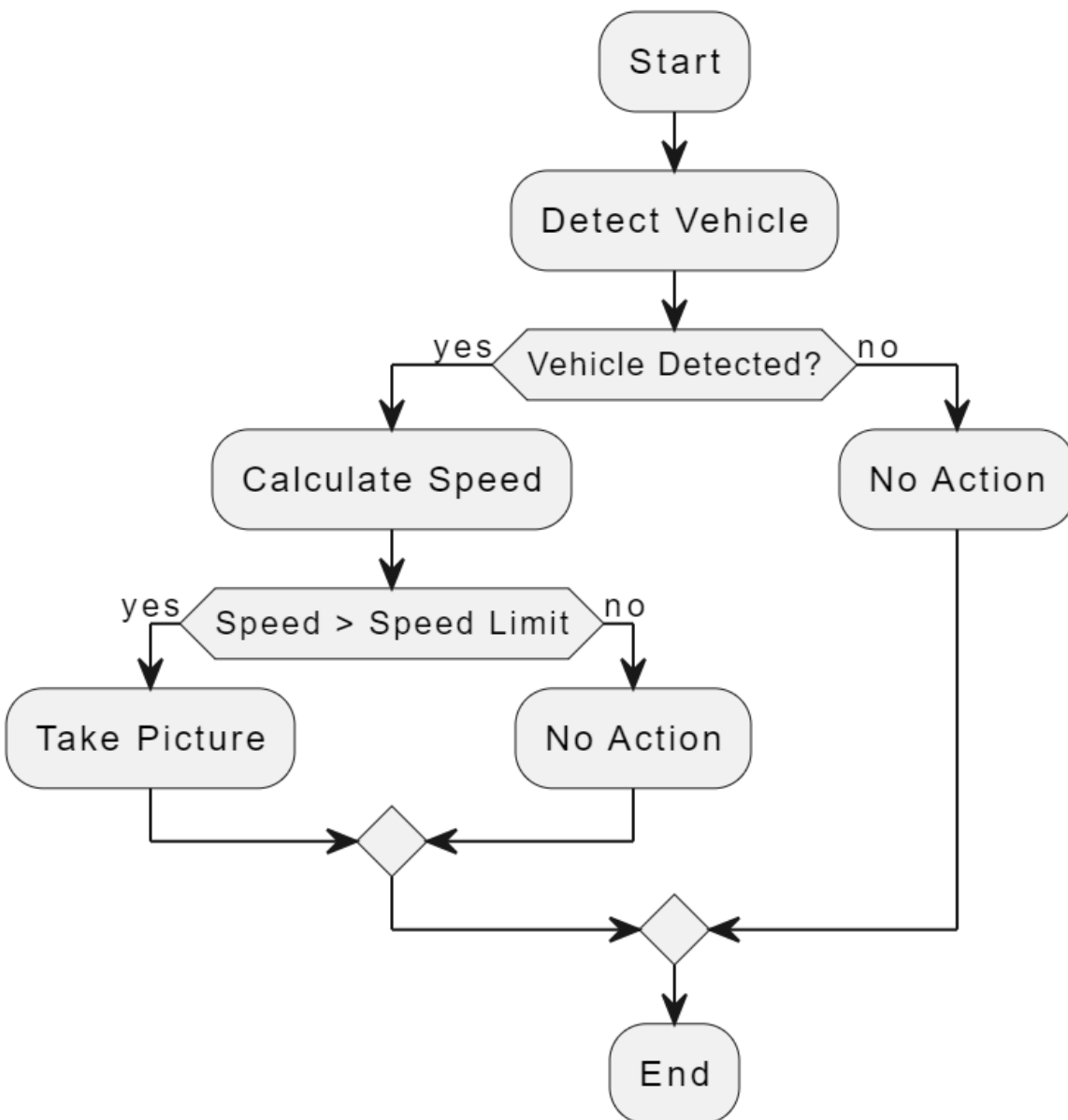


Figure 2 Activity Diagram

SEQUENCE DIAGRAM

A sequence diagram visually represents the interactions between objects or components within a system over time. It illustrates the chronological flow of messages exchanged between these elements, helping to understand how they collaborate to achieve specific tasks or functionalities.

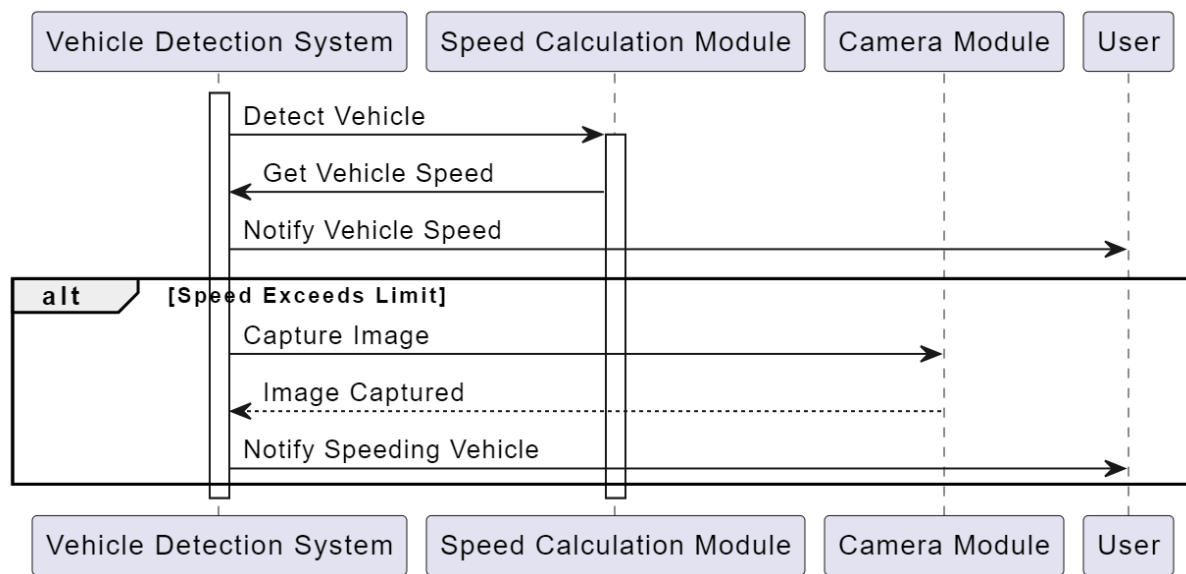


Figure 3 Sequence Diagram

5. IMPLEMENTATION

5.1 METHODOLOGY

The implementation of the "Vehicle Speed Detection" project involves a systematic approach to implementing an automated system for accurately detecting and monitoring the speed of vehicles in real-time. The project leverages computer vision techniques and advanced algorithms to process video data, track moving objects, and calculate their speeds.

In this project, the methodology that proposed has 5 stages. Which are video acquisition, region of interest making, contour detection and object detection, speed estimation, save vehicle data. The below flowchart shows the key stages in the vehicle identification and tracking process.

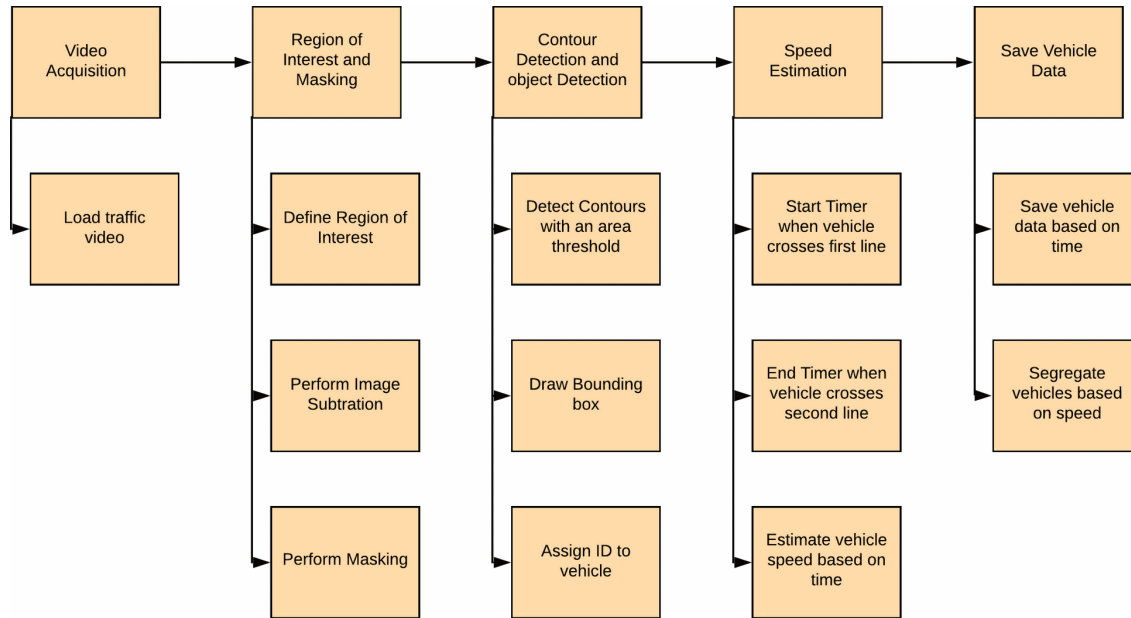


Figure 1: Flowchart

The methodology of the project integrates computer vision principles, real-time processing, and system integration to achieve an accurate and efficient vehicle speed detection system. The focus is on addressing challenges, ensuring precision, and providing a user-friendly solution that aligns with the goals of traffic management and road safety.

5.1.1. VIDEO ACQUISITION

The video used in this project is a traffic video. The number plates of the vehicle in the video are not clearly visible.



Figure 2: Video Acquisition

5.1.2. REGION OF INTEREST AND MASKING

Region of Interest (ROI) takes a smaller portion of the original video. On this ROI, Image subtraction is performed to detect a moving vehicle. (Image Subtraction helps find the difference between two frames). Masking is performed to make the moving vehicles appear white and the rest of the image black.



Figure 4 Masked image

5.1.3. CONTOUR DETECTION AND OBJECT TRACKING

Based on the area threshold of number of pixels, the contours are detected. The threshold is used to avoid detecting contours of smaller moving objects that are not vehicles. The object is tracked based on the distance between two contours between frames. An ID is assigned to each contour.

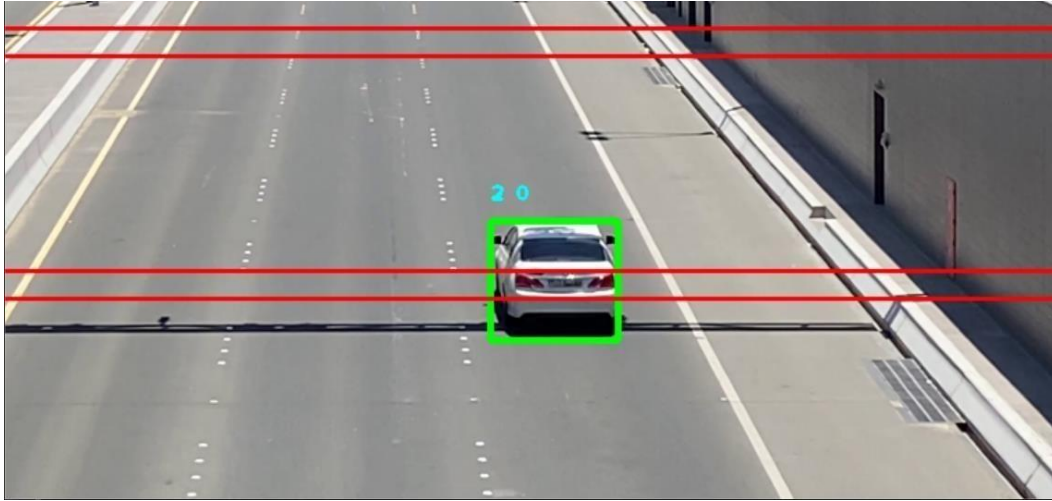


Figure 4: Contour Detection

5.1.4 SPEED ESTIMATION

Time difference between the position of a vehicle is calculated and the speed is estimated based on a formula. The timer starts when the vehicle crosses the first line, and the timer ends when the vehicle crosses the second line. The speed is displayed on top of the bounding box only when the vehicle crosses both the lines.

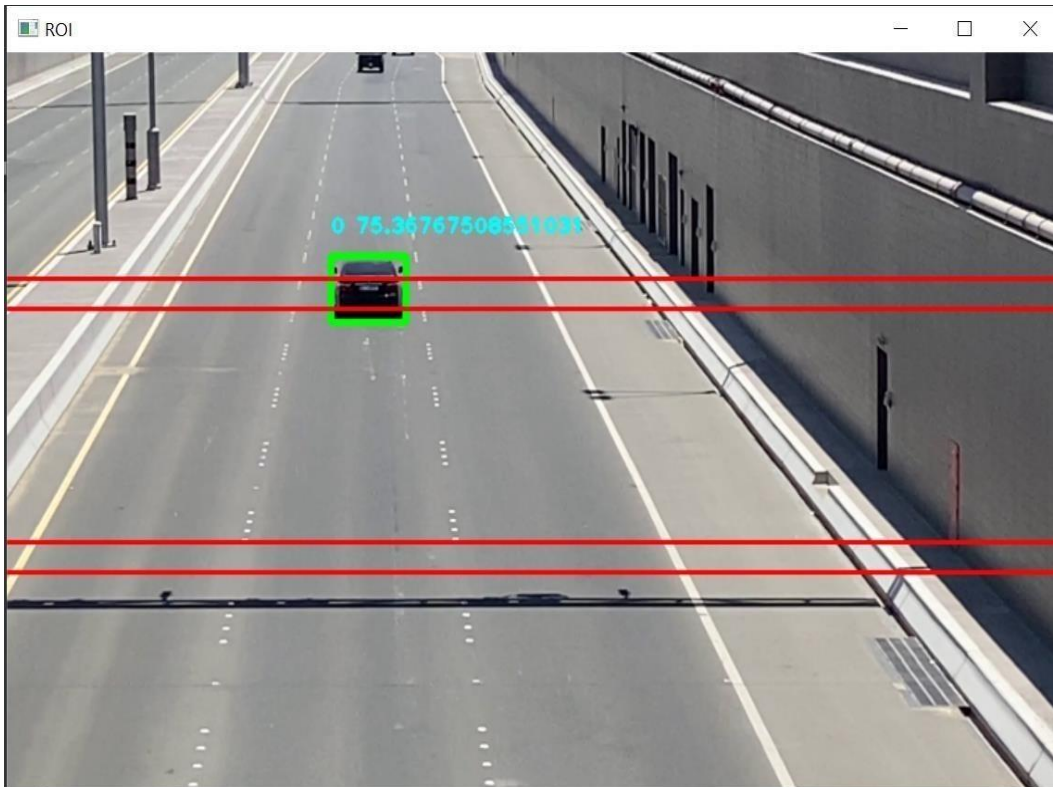


Figure 5: Speed Estimation

5.1.5 SAVE VEHICLE DATA

The picture of the bounding box (the vehicle) is saved into a file along with the speed. Vehicles crossing the speed limit is segregated into a separate folder.



Figure 6: Saved Vehicle Pictures

5.1.6. CREATE SUMMARY

The vehicle data is saved in a text file. The vehicles that exceeded the speed limit are pointed. A summary of number of vehicles and the speed violators are displayed.

```
SpeedRecord.txt - Notepad
File Edit Format View Help
ID      SPEED
-----
0        52
1        61
3        50
4       94<---exceeded
5        71
7        63
12       44
11       36
13       39
14       72
15       72
23       71
24      84<---exceeded

-----
SUMMARY
-----
Total Vehicles :      13
Exceeded speed limit : 2
```

Figure 7: Saved Summary

5.2. MODULE DESCRIPTION

5.2.1. Euclidean Distance Tracker Class (Tracker.py):

- This class implements a simple object tracker based on the Euclidean distance between object centroids.
- It maintains a dictionary of object IDs and their corresponding centroid positions.
- The `update` method takes the bounding box coordinates of detected objects and updates the tracker.
- It also handles speed calculation and recording based on predefined regions of interest (ROI) in the video frame.

5.2.2. Speed Recording and Analysis (SpeedRadar.py):

- This module records vehicle speeds and related data.
- It saves cropped images of vehicles and their speeds to a specified folder.
- The `capture` method saves images of vehicles and updates the speed record file accordingly.
- The `end` method finalizes the speed record file by adding a summary section.

5.3. INTRODUCTION TO TECHNOLOGIES USED

5.3.1. OpenCV (Open-Source Computer Vision Library):

OpenCV is a popular open-source computer vision and machine learning software library. It provides various functions and algorithms for image and video processing, including object detection, tracking, and speed estimation, which are essential components of the vehicle speed detection system.

5.3.2. NumPy:

NumPy is a fundamental package for scientific computing with Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions. NumPy is used in the script for efficient data manipulation and computation, especially in handling image data and calculations related to vehicle tracking and speed estimation.

5.3.3. Python:

Python is a widely-used programming language known for its simplicity and readability. The entire vehicle speed detection system is implemented using Python programming language, leveraging its extensive libraries and ease of development.

5.3.4. Background Subtraction Techniques:

Background subtraction is a common method used in computer vision for separating foreground objects (such as vehicles) from the background in a video stream. The script employs background subtraction techniques provided by OpenCV to detect moving vehicles in the traffic footage.

5.3.5. Euclidean Distance Calculation:

Euclidean distance is used as a metric for calculating the distance between two points in the script. It is utilized by the object tracker to measure the distance between object centroids in consecutive frames, enabling the estimation of vehicle speeds

5.4. SAMPLE CODE

Module Speed Radar Code:

```
import cv2
from tracker2 import *
import numpy as np
end = 0

#Create Tracker Object
tracker = EuclideanDistTracker()

#cap = cv2.VideoCapture("Resources/traffic3.mp4")
cap = cv2.VideoCapture("traffic4.mp4")
f = 25
w = int(1000/(f-1))

#Object Detection
object_detector =
cv2.createBackgroundSubtractorMOG2(history=None,varThreshold=None)

#KERNALS
kernalOp = np.ones((3,3),np.uint8)
kernalOp2 = np.ones((5,5),np.uint8)
kernalCl = np.ones((11,11),np.uint8)
fgbg=cv2.createBackgroundSubtractorMOG2(detectShadows=True)
kernal_e = np.ones((5,5),np.uint8)

while True:
    ret,frame = cap.read()
    if not ret:
        break
    frame = cv2.resize(frame, None, fx=0.5, fy=0.5)
    height,width,_ = frame.shape
    #print(height,width)
    #540,960

    #Extract ROI
    roi = frame[50:540,200:960]

    #DIFFERENT MASKING METHOD 2 -> This is used
    fgmask = fgbg.apply(roi)
    ret, imBin = cv2.threshold(fgmask, 200, 255, cv2.THRESH_BINARY)
    mask1 = cv2.morphologyEx(imBin, cv2.MORPH_OPEN, kernalOp)
    mask2 = cv2.morphologyEx(mask1, cv2.MORPH_CLOSE, kernalCl)
    e_img = cv2.erode(mask2, kernal_e)
```

```

contours,_ = cv2.findContours(e_img,cv2.RETR_TREE,cv2.CHAIN_APPROX_SIMPLE)
detections = []

for cnt in contours:
    area = cv2.contourArea(cnt)
    #THRESHOLD
    if area > 1000:
        x,y,w,h = cv2.boundingRect(cnt)
        cv2.rectangle(roi,(x,y),(x+w,y+h),(0,255,0),3)
        detections.append([x,y,w,h])

#Object Tracking
boxes_ids = tracker.update(detections)
for box_id in boxes_ids:
    x,y,w,h,id = box_id

if(end!=1):
    tracker.end()

cap.release()
cv2.destroyAllWindows()

```

Module Tracker Code

```

import cv2
import math
import time
import numpy as np
import os

limit = 80 # km/hr

traffic_record_folder_name = "TrafficRecord"

if not os.path.exists(traffic_record_folder_name):
    os.makedirs(traffic_record_folder_name)
    os.makedirs(traffic_record_folder_name+"//exceeded")

speed_record_file_location = traffic_record_folder_name + "//SpeedRecord.txt"
file = open(speed_record_file_location, "w")
file.write("ID \t SPEED\n-----\t-----\n")
file.close()

class EuclideanDistTracker:
    def __init__(self):

```

```

# Store the center positions of the objects
self.center_points = {}

self.id_count = 0
# self.start = 0
# self.stop = 0
self.et = 0
self.s1 = np.zeros((1, 1000))
self.s2 = np.zeros((1, 1000))
self.s = np.zeros((1, 1000))
self.f = np.zeros(1000)
self.capf = np.zeros(1000)
self.count = 0
self.exceeded = 0

def update(self, objects_rect):
    objects_bbs_ids = []

    # Get center point of new object
    for rect in objects_rect:
        x, y, w, h = rect
        cx = (x + x + w) // 2
        cy = (y + y + h) // 2

        # CHECK IF OBJECT IS DETECTED ALREADY
        same_object_detected = False

        for id, pt in self.center_points.items():
            dist = math.hypot(cx - pt[0], cy - pt[1])

            if dist < 70:
                self.center_points[id] = (cx, cy)
                objects_bbs_ids.append([x, y, w, h, id])
                same_object_detected = True

        # START TIMER
        if (y >= 410 and y <= 430):
            self.s1[0, id] = time.time()

        # STOP TIMER and FIND DIFFERENCE
        if (y >= 235 and y <= 255):
            self.s2[0, id] = time.time()
            self.s[0, id] = self.s2[0, id] - self.s1[0, id]

        # CAPTURE FLAG
        if (y < 235):
            self.f[id] = 1

    # NEW OBJECT DETECTION
    if same_object_detected is False:

```

```

        self.center_points[self.id_count] = (cx, cy)
        objects_bbs_ids.append([x, y, w, h, self.id_count])
        self.id_count += 1
        self.s[0, self.id_count] = 0
        self.s1[0, self.id_count] = 0
        self.s2[0, self.id_count] = 0

# ASSIGN NEW ID to OBJECT
new_center_points = {}
for obj_bb_id in objects_bbs_ids:
    _, _, _, _, object_id = obj_bb_id
    center = self.center_points[object_id]
    new_center_points[object_id] = center

self.center_points = new_center_points.copy()
return objects_bbs_ids

    else:
        filelet.write(str(id) + " \t " + str(sp) + "\n")
    filelet.close()

# TEXT FILE SUMMARY
def end(self):
    file = open(speed_record_file_location, "a")
    file.write("\n-----\n")
    file.write("-----\n")
    file.write("SUMMARY\n")
    file.write("-----\n")
    file.write("Total Vehicles :\t" + str(self.count) + "\n")
    file.write("Exceeded speed limit :\t" + str(self.exceeded))
    file.close()

```


6. TEST CASES

TEST CASE 1: Vehicle Speed Calculation

Input:

Video file containing vehicles moving at different speeds.

Speed limit set to 80 km/hr.

Expected Output:

Detection and tracking of vehicles.

Calculation of speeds for each vehicle.

Identification of vehicles exceeding the speed limit.

Logging of vehicle data including IDs, speeds, and images.

Display of processed video stream with speed information.

Actual Output:

Vehicles detected and tracked.

Speeds calculated for each vehicle.

Vehicles exceeding the speed limit are identified.

Vehicle data logged including IDs, speeds, and images.

Processed video stream displayed with speed information.

TEST CASE 2: Boundary Condition Testing

Input:

Video file with minimal or no traffic.

Speed limit set to 80 km/hr.

Expected Output:

Detection and tracking of any vehicles present.

No vehicles should exceed the speed limit.

Logging of vehicle data, if any.

Display of processed video stream with speed information or no vehicles detected message.

Actual Output:

Detection and tracking of any vehicles present.

No vehicles exceed the speed limit.

Vehicle data logged, if any.

Processed video stream displayed with speed information or no vehicles detected message.

TEST CASE 3: Performance Testing

Input:

Large video file with heavy traffic.

Speed limit set to 80 km/hr.

Expected Output:

Efficient detection and tracking of vehicles in real-time.

Accurate calculation of speeds for each vehicle.

Proper identification of vehicles exceeding the speed limit.

Timely logging of vehicle data.

Smooth display of processed video stream without significant lag.

Actual Output:

Efficient detection and tracking of vehicles in real-time.

Accurate calculation of speeds for each vehicle.

Proper identification of vehicles exceeding the speed limit.

Timely logging of vehicle data.

Smooth display of processed video stream without significant lag.

TEST CASE 4: Speed Limit Change

Input:

Video file containing traffic footage with varying vehicle speeds.

Speed limit parameter set to a different value (e.g., 60 km/hr) compared to the default (80 km/hr) in the script.

Expected Output:

Detection and tracking of vehicles in the video stream.

Calculation of speeds for each tracked vehicle based on the new speed limit.

Identification of vehicles exceeding the new speed limit.

Logging of vehicle data including IDs, speeds, and images based on the updated speed limit.

Display of processed video stream with bounding boxes around detected vehicles and updated speed information.

Actual Output:

Vehicles detected and tracked.

Speeds calculated for each vehicle based on the new speed limit.

Vehicles exceeding the new speed limit are identified.

Vehicle data logged including IDs, speeds, and images based on the updated speed limit.

Processed video stream displayed with updated speed information.

7. SCREENSHOTS

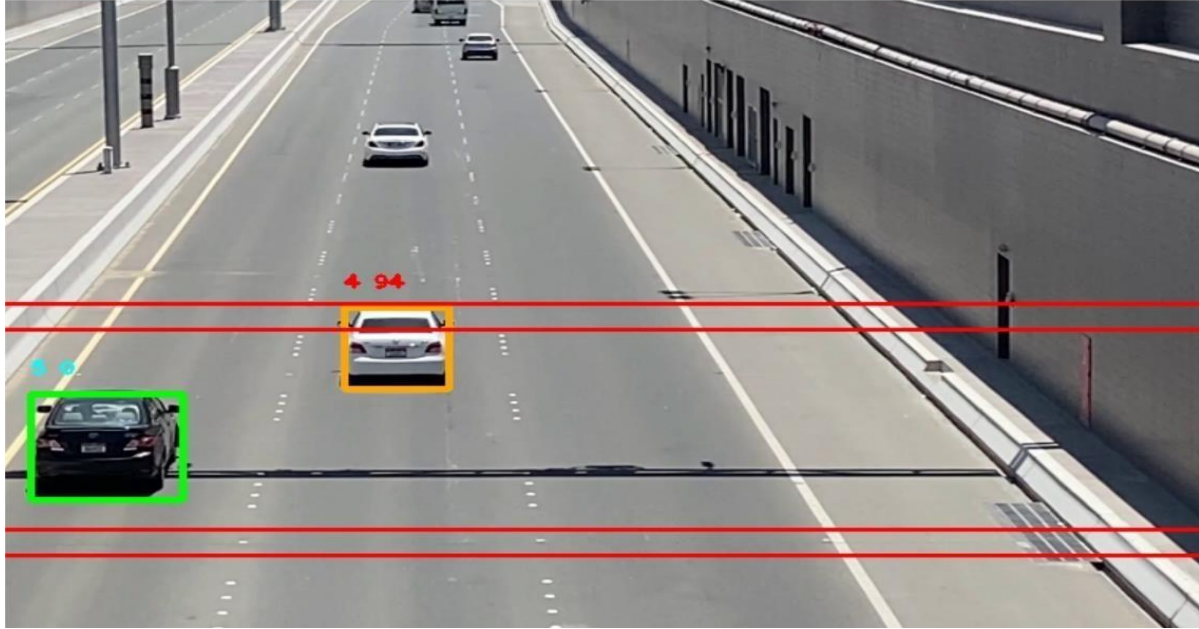


Figure 5 Speed Detection using Open CV

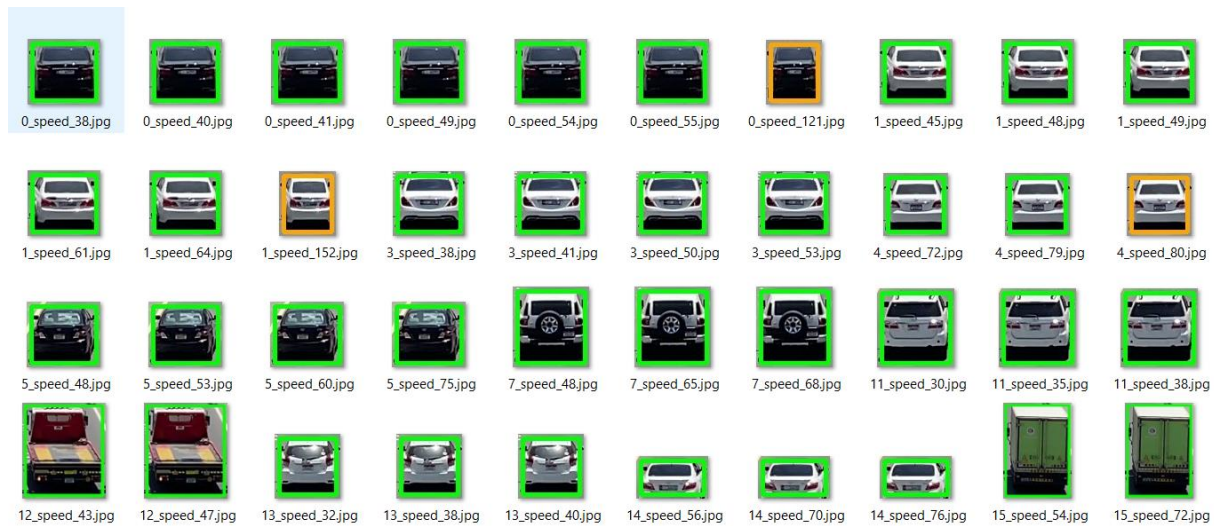


Figure 6 Images Captured by the Vehicle speed tracker

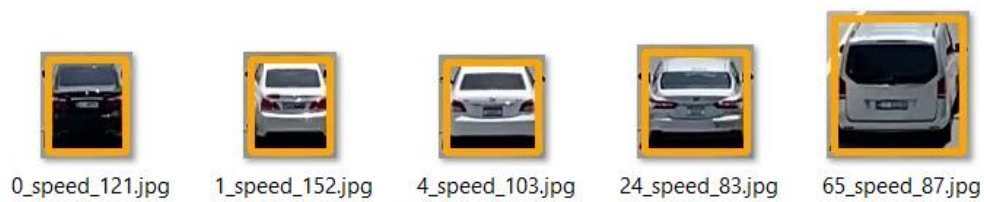


Figure 7 Images of Vehicles Exceeded the speed limit

```
SpeedRecord.txt - Notepad
File Edit Format View Help
ID      SPEED
-----
0        52
1        61
3        50
4       94<---exceeded
5        71
7        63
12       44
11       36
13       39
14       72
15       72
23       71
3 24     84<---exceeded

-----
-----
SUMMARY
-----
Total Vehicles :      13
Exceeded speed limit : 2
```

Figure 8 Speed Tracker Information

8. CONCLUSION

The project represents a sophisticated application of computer vision and object tracking techniques in the realm of traffic monitoring. By harnessing the power of OpenCV, the script adeptly captures and tracks vehicles in a video stream, offering a real-time analysis of their speeds. The Euclidean Distance Tracking algorithm facilitates precise object tracking, contributing to the system's accuracy and efficiency. Notably, the code incorporates features for capturing images of speeding vehicles, enabling visual documentation of speed limit violations. The project extends its utility by implementing a comprehensive logging algorithm, systematically recording captured images and contextual details. This logging functionality provides a structured foundation for subsequent data analysis and reporting. The code's modular design, featuring a dedicated class for object tracking, enhances its readability and maintainability. The project's significance lies in its potential applications for traffic management, safety enforcement, and data-driven decision-making, offering valuable insights into traffic patterns and contributing to the development of intelligent transportation systems.

9. FEATURE ENHANCEMENT

The "Vehicle Speed Detection" project lays a solid foundation for future developments and expansions in the field of intelligent transportation systems and computer vision applications. Several potential avenues for the project's future scope include:

1. Integration with Smart Cities:

Extend the project to integrate with broader smart city initiatives. Collaborate with traffic management systems to enhance overall urban mobility and reduce congestion.

2. Real-Time Traffic Analytics:

Implement advanced analytics to derive insights into traffic patterns, peak hours, and congestion zones. This data can be utilized for optimizing traffic signal timings and enhancing overall traffic flow.

3. Machine Learning for Anomaly Detection:

Explore the incorporation of machine learning algorithms to detect anomalous behavior on the road, such as erratic driving, lane violations, or unexpected stops, contributing to enhanced safety measures.

4. Vehicle Classification:

Enhance the system to classify vehicles based on size, type, or purpose. This information can be valuable for urban planning, infrastructure development, and targeted traffic management strategies.

5. Dynamic Speed Limit Adaptation:

Develop algorithms that dynamically adjust speed limits based on real-time traffic conditions. This can contribute to improving safety and optimizing traffic flow in varying situations.

6. Collaboration with Law Enforcement:

Collaborate with law enforcement agencies to use the system for automated speed enforcement. This could contribute to reducing the burden on manual law enforcement efforts.

By exploring these avenues, the "Vehicle Speed Detection" project can evolve into a more sophisticated and comprehensive solution for addressing the challenges and opportunities in modern traffic management and road safety.

10. BIBLIOGRAPHY

1. Afifah, Fatima, Sharmin Nasrin, and Abdul Mukit. "Vehicle speed estimation using image processing." *J. Adv. Res. Appl. Mech* 48.1 (2019): 9-16.
2. RAJU, A. NAGA, and MUGIDHI ANUSHA. "SPEED ESTIMATION AND NUMBER PLATE DETECTION USING OPEN CV." *Journal of Engineering Sciences* 14.09 (2023).
3. Quao, Tarek Nii Kpakpo. "Design and implementation of a speed detection system using radar and image processing." (2020).
4. Wafi, Zainab Nazar Khalil, R. Badlishah Ahmad, and M. P. Paulraj. "Highways traffic surveillance system (HTSS) using opencv." *2010 IEEE Control and System Graduate Research Colloquium (ICSGRC 2010)*. IEEE, 2010.
5. Kamoji, Supriya, et al. "Image processing based vehicle identification and speed measurement." *2020 International Conference on Inventive Computation Technologies (ICICT)*. IEEE, 2020.
6. ScienceDirect – 2020 - An Efficient Approach for Detection and Speed Estimation of Moving Vehicles By Tarun Kumar and Dharmender Singh Kushwaha.
7. [5] ScienceDirect – 2020 - Vehicle speed measurement model for video-based systems by Saleh Javadi , Mattias Dahl, Mats I. Pettersson..
8. ScienceDirect – 2020 - Determining Vehicle Speeds Using Convolutional Neural Networks by Alexander Grents, Vitalii Varkentin*, Nikolay Goryaev.
9. ScienceDirect – 2020 - Detection of Vehicle Position and Speed using Camera Calibration and Image Projection Methods by Alexander A S Gunawana,* , Deasy Aprilia Tanjunga, Fergyanto E. Gunawan.